

Section 2.2: Ingress Topology & Proxy Infrastructures

Enhanced Edition — Additional examples, comparisons, and interview preparation

Executive Overview

Ingress infrastructure is the first point of contact for external traffic. Understanding proxy types, load balancing algorithms, and multiplexing is crucial for designing systems that handle millions of requests while maintaining low latency.

The core mental model: Think of ingress as a series of gates, each operating at a different OSI layer. The further up the stack you go, the more intelligence you can apply — but at higher CPU cost and added latency. A well-designed ingress stack uses the simplest gate that solves each problem.

Proxy Architectural Delineation

Forward Proxy vs. Reverse Proxy

The single most important distinction: a forward proxy serves the **client's** interests; a reverse proxy serves the **server's** interests.

Forward Proxy (client-side, client is aware):

```
Client → Forward Proxy → Internet → Target Server
      ↑
      Client configures this explicitly
      (system proxy setting, browser proxy, PAC file)

The target server sees the proxy's IP, not the client's
```

Reverse Proxy (server-side, client is unaware):

```
Client → Reverse Proxy → Backend Servers
      ↑
      Client thinks it's talking to the origin
```

Server topology is completely hidden
Client sees only the proxy's IP/domain

Where Each Lives in Real Architectures

Forward proxy real-world scenarios:

Use Case	Example	What the Proxy Does
Corporate internet control	Squid, Zscaler	Blocks social media, logs URLs, enforces DLP
Anonymous browsing	Tor exit node	Hides origin IP from target server
Bandwidth caching	ISP transparent proxy	Caches CDN content to reduce upstream costs
Security scanning	Cloudflare Gateway	Inspects outbound TLS via MITM (with root cert)

Reverse proxy real-world scenarios:

Use Case	Example	What the Proxy Does
Load balancing	AWS ALB, NGINX	Distributes requests across app servers
TLS termination	AWS ACM + ALB	Decrypts HTTPS; backend gets HTTP
Rate limiting	Kong, NGINX Plus	Rejects requests exceeding per-IP/key limits
CDN edge caching	Cloudflare, Fastly	Serves static assets from cache
A/B routing	Envoy, Istio	Sends 5% traffic to canary
Web Application Firewall	Cloudflare WAF	Blocks SQLi, XSS at edge

The Confused Middle: Transparent Proxies

A transparent proxy intercepts traffic without client configuration — common in ISPs and enterprise networks:

Client → [Traffic silently intercepted] → Proxy → Internet

Client doesn't know the proxy exists

Proxy may:

- Cache responses (ISP bandwidth saving)
- Inspect HTTP (not HTTPS — would require cert injection)
- Block blacklisted domains

```
Detection: curl -v http://example.com | look for Via or X-Forwarded-For headers
```

Edge Routing Infrastructures

Load Balancer Algorithms — the Selection Guide

The choice of algorithm determines how load distributes across backends, and which algorithm is right depends on whether your requests have equal cost.

Round Robin:

```
Request 1 → Server A  
Request 2 → Server B  
Request 3 → Server C  
Request 4 → Server A (repeats)
```

Best for: Stateless services with uniform request cost

Weakness: 100ms and 10s requests hit same server equally — expensive requests starve others

Weighted Round Robin:

```
Server A (weight 3) → gets 3 requests per cycle  
Server B (weight 1) → gets 1 request per cycle
```

Best for: Heterogeneous servers (different CPU/RAM) or canary deployments

Example: New server at 10% weight during rollout

Least Connections:

```
Server A: 50 active connections  
Server B: 12 active connections ← next request goes here  
Server C: 34 active connections
```

Best for: Long-lived connections (WebSockets, gRPC streams, DB connections)

Weakness: Connection count doesn't equal CPU load

Least Response Time (most intelligent):

```
Server A: avg 250ms response time  
Server B: avg 40ms response time ← next request goes here  
Server C: avg 180ms response time
```

Best for: Heterogeneous backends, variable request cost
Used by: Envoy (with P2C – Power of Two Choices algorithm)

Consistent Hashing:

`hash(user_id) % ring_position` → always routes to same server

Request for `user_id=42` always → Server B (barring failure)

Best for: Stateful services, caches (maximize cache locality)
Minimizes cache invalidation when servers are added/removed
Used by: Memcached, DynamoDB, Cassandra ring

Virtual nodes: each physical server owns multiple ring positions
→ more uniform distribution, smoother redistribution on failure

IP Hash:

`hash(client_ip)` → always routes to same server

Best for: Session affinity (sticky sessions) without shared session store
Weakness: IP changes (mobile, VPN) break affinity
CDN/NAT means many users share one IP → uneven distribution

Load Balancing Algorithm Comparison – Decision Matrix:

Algorithm	Complexity	Best Use Case	Weakness	When to Use
Round Robin	$O(1)$	Stateless APIs, uniform requests	Ignores backend load	Default for simple services
Weighted RR	$O(1)$	Canary deploys, heterogeneous hardware	Still ignores actual load	Rolling out new versions
Least Connections	$O(\log n)$	Long-lived connections	Connection count $\neq$ load	WebSockets, gRPC streams
Least Response Time	$O(\log n)$	Variable request cost	Requires health monitoring	Production recommendation
Consistent Hashing	$O(\log n)$	Caching, stateful services	Requires stable ring	Redis cluster, session stores
IP Hash	$O(1)$	Sticky sessions	IP changes break affinity	Legacy session systems

Algorithm	Complexity	Best Use Case	Weakness	When to Use
Random	O(1)	Stateless, high scale	No intelligence	Extremely simple cases

Algorithm Selection Framework:

```

Stateful service (sessions, cache)?
└ Yes → Consistent Hashing (with virtual nodes)

Long-lived connections (WebSocket, SSE)?
└ Yes → Least Connections

Variable request cost (mix of cheap + expensive)?
└ Yes → Least Response Time

Canary deployment (routing % of traffic)?
└ Yes → Weighted Round Robin

None of the above (simple stateless API)?
└ Least Response Time (if available) OR Round Robin

```

Real production examples: - **Netflix (Envoy):** Least Response Time with P2C (Power of Two Choices) — samples 2 random backends, picks the better one - **Twitter:** Consistent Hashing for cache routing to maximize hit rate - **Stripe API:** Round Robin for stateless API endpoints with circuit breaking per backend

API Gateways vs. Load Balancers — the Crisp Distinction

These are frequently confused in interviews. The key difference: a load balancer distributes traffic; an API gateway governs it.

Load Balancer (L4 or L7):

```

Input: TCP connection or HTTP request
Output: Same request forwarded to a chosen backend
Added value: Routing decision + connection pooling + health checks
Cannot do: Authentication, rate limiting per user, request transformation

```

API Gateway (always L7):

```

Input: HTTP request with headers, body, auth tokens
Output: Modified request forwarded to backend (or rejected)
Added value: Auth, rate limiting, transformation, logging, versioning

```

Can do: Token validation, body rewrite, protocol translation (REST→gRPC)
Cannot do: L4 passthrough (it must parse HTTP)

Architecture integration with both:

```
Internet
  ↓
DNS (GeoDNS for region routing)
  ↓
Global Load Balancer (AWS Route 53 + CloudFront)
  ↓
Regional L4 NLB (handles TCP, lowest latency)
  ↓
Regional L7 ALB (HTTP routing by path/host)
  ↓
API Gateway (Kong/Apigee: auth, rate limit, transform)
  ↓
Microservices (via service mesh: Istio/Linkerd for internal LB)
  ↓
Databases / downstream
```

When to use each:

Scenario	Use	Reason
Balancing 10 app servers	L7 ALB only	No auth needed; path routing sufficient
Public API with keys	API Gateway + ALB	Key validation, per-key rate limits
Internal microservices	Service mesh (Envoy)	mTLS, circuit breaking, observability
TCP pass-through (non-HTTP)	L4 NLB	Cannot parse non-HTTP at L7
Gaming server (UDP)	L4 NLB	UDP not supported by L7 LBs

API Gateway vs Load Balancer — Feature Comparison:

Feature	L4 Load Balancer (NLB)	L7 Load Balancer (ALB)	API Gateway (Kong/Apigee)
OSI Layer	Layer 4 (TCP/UDP)	Layer 7 (HTTP)	Layer 7 (HTTP)
Protocol awareness	No (just forwards packets)	HTTP only	HTTP, gRPC, GraphQL
	~0.1ms	~1-2ms	~2-5ms

Feature	L4 Load Balancer (NLB)	L7 Load Balancer (ALB)	API Gateway (Kong/ Apigee)
Latency overhead			
Path-based routing	✗	✓ <code>/api/users → UserService</code>	✓ Advanced routing rules
Authentication	✗	✗	✓ JWT, OAuth, API keys
Rate limiting	✗	Basic (per IP)	✓ Per user/key, quota management
Request transformation	✗	✗	✓ Add headers, rewrite body
Protocol translation	✗	✗	✓ REST → gRPC, SOAP → REST
Analytics/ logging	Basic (connection counts)	HTTP logs	✓ Full request/response capture
Plugins/ extensibility	✗	✗	✓ Custom logic via plugins
Cost	Low (\$0.006/hour)	Medium (\$0.023/hour)	High (\$0.10+/hour + per-call)

When to skip API Gateway and use ALB directly: - Internal services (no public auth needed) - Cost-sensitive applications - Ultra-low latency requirements (<5ms) - Simple routing without transformation

When API Gateway is worth the cost: - Public APIs with diverse clients - Need per-customer quotas/throttling - Complex auth flows (multi-tenant) - Protocol translation requirements - Monetized APIs (need metering)

Health Checks — Active vs. Passive

Active health check (LB polls backend):

```

Every 10s: GET /health HTTP/1.1 → Backend
If 200: backend is healthy, keep in rotation
If timeout/5xx: mark unhealthy, remove from rotation

Pros: Detects failure before real traffic hits bad backend
Cons: Overhead (adds load), may not reflect real request behavior

```

Passive health check (LB observes real traffic):

```
Observes response codes from real requests:  
If > 50% of last 10 requests to Server A return 5xx:  
    Mark Server A unhealthy  
  
Pros: Zero overhead; reflects real behavior  
Cons: Real users hit failures before backend is removed
```

Production pattern: Use both — active to detect dead hosts quickly, passive (outlier detection) to catch degraded-but-not-dead hosts.

Traffic Routing Models

Unicast, Broadcast, Multicast, Anycast — the Full Picture

Unicast (99% of internet traffic):

```
One source → One specific destination  
Each client requires its own stream even for identical content  
Cost: O(viewers) bandwidth from origin  
  
Example: Amazon video to your browser = unicast  
Your stream is separate from your neighbor's identical stream
```

Broadcast (LAN-only, never internet-routed):

```
Source → All hosts on subnet  
Limited to L2 domain (Ethernet broadcast domain)  
255.255.255.255 = limited broadcast (own subnet only)  
  
Used by: DHCP discovery (client doesn't know server IP yet),  
         ARP (who has IP X? Tell 192.168.1.1)  
  
Internet impact: Routers block broadcast — it never crosses a subnet
```

Multicast (one-to-many with receiver opt-in):

```
Source → Only subscribed receivers (IGMP join)  
Network switches maintain group membership tables  
Bandwidth scales with number of links, not receivers  
  
Ideal for: Stock price feeds (10K receivers, 1 stream),  
          Live video to millions (IPTV on managed networks),
```


VRRP/OSPF protocol traffic between routers

Limitation: Requires network infrastructure support (IGMP snooping on switches)
Not available on public internet – CDN multicast is a myth

Anycast (one-to-nearest):

Multiple servers advertise the SAME IP prefix via BGP from different locations

Client DNS query → Hits nearest BGP-advertised node (BGP routing selects shortest AS path)

Example: 1.1.1.1 (Cloudflare DNS):

User in Singapore → Singapore edge node

User in London → London edge node

Same IP, different physical servers

Benefits:

- Automatic load distribution by geography
- DDoS absorption: attack volume spreads across all PoPs
- Failover: if Singapore PoP dies, BGP re-routes to next nearest

Used by: Cloudflare, Google (8.8.8.8), DNS root servers (a.root-servers.net), AWS Route 53, Akamai, Fastly

Anycast vs. GeoDNS — a subtle distinction interviewers probe:

	Anycast	GeoDNS
How routing works	BGP routes packets to nearest PoP	DNS returns different IP per region
Failover speed	Seconds (BGP reconvergence)	Seconds to minutes (DNS TTL)
Session continuity	Same IP always → connection can persist	Different IP → new TCP connection
Use case	DNS, UDP services	HTTP services with CDN

Multiplexing Mechanics

HTTP Evolution — the Full Picture with Numbers

HTTP/1.0 (1996):

For a webpage with 50 resources:
 $50 \text{ TCP connections} \times (80\text{ms RTT handshake} + 80\text{ms request}) = 8 \text{ seconds}$
(Yes, early websites were painfully slow)

HTTP/1.1 Keep-Alive (1997):

Reuses one TCP connection for all requests — but serially:

Request A: 100ms
Request B: 200ms (waits for A to complete)
Request C: 50ms (waits for A+B)
Total: 350ms

Browser hack: Opens 6 parallel connections to same host

- 6× parallelism, but 6× connection overhead
- Creates thundering herd on server (sudden 6× connection spike)

HTTP/2 Multiplexing (2015):

Single connection, streams interleaved:

Stream 1: GET /api/users	————→	200ms
Stream 2: GET /api/orders	———→	80ms
Stream 3: GET /api/cart	————→	140ms

↑

All in parallel on ONE TCP connection

Total: $\max(200, 80, 140) = 200\text{ms}$ (vs 350ms serial)

HTTP/2 server push (pre-loads resources):

Client: GET /index.html

Server: Here's index.html + CSS + JS you'll need (pushed proactively)

TCP HoL blocking remains:

Packet loss on connection → ALL streams pause

(TCP must deliver bytes in order; lost packet blocks the receive buffer)

HTTP/3 + QUIC (2022 RFC standardized):

UDP-based, per-stream reliability:

Stream 1: GET /api/users	————→	200ms
Stream 2: GET /api/orders	———→	80ms

[Packet loss on Stream 1's data]

Stream 2: Still delivers normally ← KEY DIFFERENCE FROM HTTP/2

Stream 1: Retransmits only its own lost packets

0-RTT for reconnection:

Client has session ticket from previous connection

Sends data in first packet, no handshake

Connection migration:

Phone switches WiFi → 5G → QUIC continues on new IP

TCP would need new connection + handshake

Deployment challenge:

UDP port 443 often blocked by enterprise firewalls

Solution: fallback to HTTP/2 if QUIC blocked

HTTP Protocol Trade-Off Matrix — Extended

Protocol	Head-of-Line Blocking	Handshake Cost	Multiplexing	Connection Migration	Best Use Case
HTTP/1.1	Per-request (serial)	1-2 RTT	No (6 parallel conns)	No	Legacy clients
HTTP/2	TCP-level	1-2 RTT	Yes (streams)	No	Most web apps today
HTTP/3	None	0-1 RTT	Yes (streams)	Yes	Mobile, high-loss networks
gRPC (HTTP/2)	TCP-level	1-2 RTT	Yes (streaming)	No	Internal microservices
WebSocket	None (full-duplex)	1 RTT + upgrade	N/A	No	Real-time bidirectional

WebSocket vs. Server-Sent Events vs. Long Polling

Real-time delivery mechanisms are a common interview topic — understanding when to pick each is essential.

Long Polling (hack on HTTP/1.1):

Client: GET /events (holds connection open)

Server: ... waits until event occurs ...

Server: 200 OK { event: "new_message" }

Client: immediately sends GET /events again

Pros: Works everywhere; no special infrastructure

Cons: One connection per client; high overhead; latency = server response time
Use: Legacy fallback; Jira, old Slack clients

Server-Sent Events (SSE):

Client: GET /events (persistent HTTP/1.1 connection)
Server: streams event: data\n\n formatted messages indefinitely

data: {"type":"price","value":150.23}
data: {"type":"price","value":150.45}

Pros: Simple; built-in reconnect; HTTP-compatible (works through proxies)
Cons: Server-Client only (unidirectional); one stream per client
Use: Stock tickers, live scores, notification feeds

WebSocket (full-duplex):

1. HTTP Upgrade:
GET /chat HTTP/1.1
Upgrade: websocket
Connection: Upgrade

2. After upgrade: raw bidirectional frames on same TCP connection
Client → Server: {"msg":"hello"}
Server → Client: {"msg":"world"}

Pros: Bidirectional; low overhead (2-byte frame header vs full HTTP headers)
Cons: Stateful (load balancer needs sticky sessions or shared state);
firewalls may block non-standard ports
Use: Chat (Slack, Discord), collaborative editing (Figma), gaming, trading terminals

Interview Questions

★ Load Balancing Algorithm Selection

You are designing a load balancer for a machine learning inference service where 95% of requests complete in 20ms but 5% require GPU processing and take 2 seconds. Which algorithm do you choose and why?

Solution Framework

Why round-robin fails here:

Round-robin sends equal requests to all servers:

Server A: receives 100 RPS; 5 are 2-second GPU jobs

Server A: 5 concurrent 2s requests + 95 fast ones

After 2 seconds: Server A has accumulated 10+ long requests

Round-robin still sends it $100/3 = 33$ more requests

Server A becomes increasingly overloaded while B and C are idle

Why least-connections is better but still imperfect:

Server A: 8 connections (5 GPU, 3 fast)

Server B: 2 connections ← next request routed here

Server C: 3 connections

Problem: 8 connections on A may be 8 fast requests (160ms total CPU)

vs 3 connections on B which are 3 GPU requests (6 seconds each)

Connection count ≠ processing load for bimodal workloads

Best choice: Least Response Time (with active connection cap):

Server A: avg response 400ms (degraded by GPU queue)

Server B: avg response 21ms ← route here

Server C: avg response 180ms

Result: Routes away from servers backing up under GPU load

Naturally rebalances as GPU jobs complete

Implementation with Envoy:

load_assignment:

endpoints:

lb_policy: LEAST_REQUEST # Uses response time + active requests

Additional safeguard: max_connections per backend

If Server A hits 50 active connections → circuit-break, return 503

Forces load shedding before server OOMs

Alternative: Request routing by type:

Option: Two separate backend pools

GET /infer/fast → Pool A (CPU-only, 10 servers, 20ms)

GET /infer/gpu → Pool B (GPU-enabled, 3 servers, 2s)

Requires: Client indicates job type (query param or header)

Benefit: GPU pool auto-scales independently from fast pool

This is how ML serving systems like TorchServe and Triton work:

"batched" vs "real-time" inference queues

Key insight: Bimodal latency distributions break simple algorithms. Least response time is the robust default; two-tier routing is optimal if you can classify requests upfront.

☆☆ Distributed Rate Limiting

Your API gateway handles 100K RPS across 20 instances. Implement rate limiting that prevents any single user from exceeding 1,000 requests per minute without overwhelming Redis.

Solution Framework

The fundamental tension:

```
Perfect global rate limiting → every request hits Redis → Redis becomes the
bottleneck
No coordination → each instance allows 1,000 RPM → total 20,000 RPM per user

Solution space: trade accuracy for throughput
```

Algorithm 1 — Fixed Window Counter (simplest, has boundary exploit):

```
def check_limit(user_id: str, limit: int = 1000) -> bool:
    key = f"rate:{user_id}:{int(time.time()) // 60}" # 1-minute window
    count = redis.incr(key)
    if count == 1:
        redis.expire(key, 120) # 2 minutes for safety
    return count <= limit

# Exploit: User sends 1000 at 00:59 and 1000 at 01:00
# = 2000 requests in 2 seconds (window boundary)
```

Algorithm 2 — Sliding Window Log (accurate, memory-heavy):

```
def check_limit(user_id: str, limit: int = 1000) -> bool:
    now = time.time()
    window_start = now - 60 # 60-second sliding window
    key = f"rate_log:{user_id}"

    pipe = redis.pipeline()
    pipe.zremrangebyscore(key, 0, window_start) # remove old timestamps
    pipe.zadd(key, {str(now): now})             # add current request
    pipe.zcard(key)                             # count in window
    pipe.expire(key, 120)
    _, _, count, _ = pipe.execute()

    return count <= limit
```

```
# Accurate but: stores one entry per request = 1000 entries per user
# At 100K RPS: 100K Redis entries/second, significant memory
```

Algorithm 3 — Sliding Window Counter (best balance):

```
def check_limit(user_id: str, limit: int = 1000) -> bool:
    now = int(time.time())
    current_window = now // 60
    prev_window = current_window - 1
    seconds_into_window = now % 60

    cur_key = f"rate:{user_id}:{current_window}"
    prev_key = f"rate:{user_id}:{prev_window}"

    cur_count = int(redis.get(cur_key) or 0)
    prev_count = int(redis.get(prev_key) or 0)

    # Weight previous window by remaining fraction
    prev_weight = (60 - seconds_into_window) / 60
    effective_count = cur_count + (prev_count * prev_weight)

    if effective_count < limit:
        redis.incr(cur_key)
        redis.expire(cur_key, 120)
        return True
    return False

# Max error: ~0.003% over-allowance (acceptable)
# Memory: 2 keys per user vs 1000+ for log approach
```

Algorithm 4 — Local Cache + Async Sync (for 100K RPS at scale):

```
Per gateway instance (in-memory):
- Local token bucket per user
- Capacity: limit / num_instances = 50 RPM per instance
- Refill rate: 50 tokens/minute

Every 1 second: sync with Redis
- Report local consumed tokens
- Receive global adjustment:
    if global_remaining < local_budget:
        local_budget = global_remaining / num_instances

Result:
- 0 Redis calls on hot path (in-memory only)
- 1 Redis call per second per instance (20 calls/sec total)
- Accepts up to 1 second of over-budget requests
- Redis load: 20 ops/s instead of 100,000 ops/s (5000× reduction)
```

Redis Lua for atomic sliding window (prevents race conditions):

```

local key = KEYS[1]
local now = tonumber(ARGV[1])
local window = tonumber(ARGV[2])
local limit = tonumber(ARGV[3])

redis.call('ZREMRANGEBYSCORE', key, 0, now - window)
local count = redis.call('ZCARD', key)
if count < limit then
    redis.call('ZADD', key, now, now .. '-' .. math.random())
    redis.call('EXPIRE', key, window + 1)
    return 1
end
return 0

```

Recommendation for 100K RPS:

```

Use Algorithm 4 (local cache + async sync):
  Redis load: ~20 ops/second (from 20 instances × 1 sync/second)
  Accuracy: 99%+ (1-second window for over-budget requests)
  Failure mode: Redis down → fall back to local limits only
                (never take down your API because rate limiter is unavailable)

Monitor: rate_limiter_redis_latency_p99 < 5ms
         rate_limiter_local_decisions_pct > 99%

```

Key insight: Perfect global rate limiting at 100K RPS requires coordination that becomes the bottleneck. Choose approximate algorithms — a 1% overage on rate limits is acceptable; a 10ms added latency on every request is not.

☆☆☆ CDN and Multi-Region Ingress Design

Design the ingress architecture for a global e-commerce platform serving 5M RPS at peak (Black Friday). Users are in North America, Europe, and Asia-Pacific. Explain your CDN strategy, anycast routing, and failover behavior.

Solution Framework

Traffic and geography analysis:

```

5M RPS total:
  NA: 50% = 2.5M RPS (peak Eastern time)
  EU: 30% = 1.5M RPS (peak CET business hours)
  APAC: 20% = 1.0M RPS (peak JST/SGT business hours)

Black Friday pattern: 10× normal → 5M RPS is the peak
Normal load: ~500K RPS

```


Content classification — what to cache where:

Static assets (~70% of requests):

- JS, CSS, images, fonts — identical for all users
- CDN cache TTL: 1 year (cache-busted by hash in filename)
- Cache hit rate target: >99%
- Never reaches origin for cached content

Product pages (~20% of requests):

- Personalized elements: cart count, wishlist status
- Non-personalized: product title, images, price, reviews
- Strategy: Edge Side Includes (ESI) or partial caching
- Cache the product "shell" (5 min TTL)
- Inject personalized elements client-side (JS fetch)
- Cache hit rate: ~70% (after personalization extracted)

Checkout / account (~10% of requests):

- Fully personalized, session-dependent
- Cannot cache — must hit origin
- Optimize: CDN accelerates connection (anycast edge → origin TCP tunnel)

Anycast routing for global acceleration:

Approach: Cloudflare / Akamai / Fastly with anycast IP

User in Singapore: DNS resolves to 104.16.0.0/12 (Cloudflare)

BGP routes to nearest PoP: Singapore PoP

- Singapore PoP → Cache hit: responds locally (1-5ms)
- Cache miss: PoP tunnels to nearest origin via optimized backbone (not public internet)

vs. No CDN:

User in Singapore → public internet → US origin

= 200ms+ RTT + congestion on public internet cross-Pacific

CDN backbone advantage:

- Cloudflare has private fiber/leased lines between PoPs
- Private backbone RTT SG→US: ~100ms (vs 180ms+ public)
- TCP connection reuse (HTTP/2 multiplexed) × 1000s of users

Regional origin architecture (active-active):

3 origin regions: us-east-1, eu-west-1, ap-southeast-1

GeoDNS (Route 53 latency routing):

CDN edge → origin: routes to lowest-latency origin

Singapore CDN PoP → ap-southeast-1 (15ms) ← normal

Singapore CDN PoP → us-east-1 (120ms)

Singapore CDN PoP → eu-west-1 (160ms)

Origin data replication:

- Product catalog: replicated read replicas in each region (RDS read replica)
- User carts: global DynamoDB table with regional replicas
- Inventory: eventual consistency OK for display; strong consistency at checkout only
- Orders: single-region primary (us-east-1) with async replication

Failover behavior:

Scenario 1: ap-southeast-1 origin fails

- CDN health check detects origin failure (active HTTP health probe every 30s)
- Route 53 health-check routing fails over to us-east-1
- All APAC CDN cache misses now route to us-east-1
- Latency increase: 15ms → 120ms for cache misses
- Cache hit rate: 70%+ → most users unaffected

Scenario 2: Single CDN PoP (Singapore) fails

- BGP route for Singapore PoP withdrawn
- User traffic re-routes to next nearest PoP (Hong Kong, Tokyo)
- Latency increase: 5ms → 30-50ms
- Fully transparent to users

Scenario 3: CDN provider outage (rare but happened: Fastly 2021, Cloudflare 2022)

- Multi-CDN strategy: primary CDN + secondary CDN (different provider)
- GeoDNS or DNS failover switches to secondary within 60s (DNS TTL = 60s)
- Cost: ~20% overhead for redundant CDN contracts

Rate limiting at the edge (Black Friday DDoS protection):

Layer 1 (CDN level – Cloudflare WAF):

- Block: >1000 RPS from single IP (bot mitigation)
- Challenge: >100 RPS from single IP (CAPTCHA challenge)
- Allow: Legitimate users (pattern analysis)

Layer 2 (API Gateway – per API key):

- Public API: 100 RPM per key
- Authenticated users: 1000 RPM
- Business accounts: 10K RPM

Layer 3 (Origin – circuit breaker):

- If origin response time > 2s for 10% of requests: serve cached stale content
- If origin error rate > 5%: enable "graceful degradation" mode
 - Cart shows last known state
 - Checkout requires re-auth against live origin only

Cost modeling:

- CDN bandwidth: 5M RPS × 50KB avg response × 70% cache hit = 175 GB/s CDN egress
- CDN egress at \$0.02/GB: ~\$300K/month
- Without CDN (origin egress at \$0.09/GB): ~\$1.35M/month

CDN cost saving: ~\$1M/month → CDN pays for itself many times over

For Black Friday specifically:

CDN absorbs 10× traffic spike without any origin scaling

Origin capacity stays at 1.5M RPS (500K × 3 regions)

Without CDN: would need 10× origin capacity for 1 day/year

Key insights: 1. Anycast routing routes users to the nearest CDN PoP automatically via BGP — no application-level routing needed. 2. Separate caching strategies for static (99%+ hit rate), product pages (70%), and personalized content (0%) — never apply one TTL to everything. 3. Multi-CDN is the last line of defense against provider outages — worth the cost for critical infrastructure. 4. Black Friday proves the architecture: if CDN absorbs 95% of traffic, your 5× peak capacity at origin handles the remaining dynamic requests.

Quick Reference: Ingress Interview Cheat Sheet

When asked about...	Key points to hit
Forward vs. reverse proxy	Forward: client-side, hides client; reverse: server-side, hides servers
Load balancing algorithms	Round-robin (uniform cost), Least Connections (long-lived), Consistent Hash (stateful), Least Response Time (variable cost)
API Gateway vs. Load Balancer	LB distributes; Gateway governs (auth, rate limit, transform)
Anycast	Same IP from multiple locations; BGP routes to nearest; used by DNS, CDN
HTTP/2 vs. HTTP/3	HTTP/2: TCP HoL blocking; HTTP/3: per-stream reliability, 0-RTT, connection migration
Rate limiting	Fixed window (boundary exploit); Sliding window (accurate); Token bucket (burst-friendly); Local cache + async sync (scalable)
Health checks	Active (probing) vs. passive (observing real traffic); use both
WebSocket vs. SSE	WS: bidirectional, stateful; SSE: server→client only, HTTP-compatible